



PASSIVE SECOND BRAIN

AI-Powered Automatic Knowledge Graph Construction
from Digital Learning Activity

Document Type	Product Requirements Document (PRD)
Project Domain	CSE – Artificial Intelligence & Machine Learning
Academic Year	2025–2026 Final Year B.Tech
Version	v1.0
Prepared By	Prasad
Institution	KIT's College of Engineering

1. Executive Summary

Passive Second Brain is a final-year B.Tech capstone project in Artificial Intelligence and Machine Learning. The system addresses a fundamental and largely unsolved problem in personal knowledge management: the automatic, passive construction of a semantic knowledge graph from a user's everyday digital learning activity.

Unlike all existing tools — Notion, Obsidian, Roam Research, Readwise — which require manual content capture and tagging, Passive Second Brain operates entirely in the background. The user simply lives their digital life — browsing articles, watching educational videos, opening PDFs — and the system silently ingests, processes, and structurally organizes that knowledge into a living, queryable graph.

 One-Line Pitch:
<i>"Passive Second Brain is an AI system that silently watches how you learn,</i>
<i>automatically builds a semantic map of your knowledge, and ensures nothing you</i>
<i>ever studied is forgotten — requiring zero effort from the user."</i>

2. Pain Points

The following five pain points were identified through analysis of existing knowledge management tools, student behaviour patterns, and cognitive science research on human memory:

P1 – Knowledge Evaporation

Students and professionals consume large volumes of digital content daily – research papers, video lectures, articles, documentation – yet retain less than 10% of it after 30 days due to the natural forgetting curve (Ebbinghaus, 1885). There exists no automated system to counteract this at the point of consumption.

P2 – Manual Capture Friction

Existing knowledge management tools such as Notion, Obsidian, and Roam Research require the user to manually save, tag, and link every piece of content they consume. This creates cognitive overhead that causes most users to abandon the system within weeks. The tool that is supposed to reduce mental load itself becomes a burden.

P3 – Isolated Knowledge Silos

Even when content is saved, it is stored as flat, disconnected files. A user who studied transformer architecture in August and reads about RAG systems in November receives no signal that these two concepts are deeply related. Knowledge never compounds – it accumulates without connecting.

P4 – No Temporal Awareness

Current tools have no memory of when something was learned or how long ago it was last revisited. There is no mechanism to alert the user that a critical concept is fading from memory before it becomes completely forgotten.

P5 – Zero Passive Intelligence

Every existing solution is fully reactive – it only works when the user actively interacts with it. There is no system that observes a user's natural digital learning behaviour and builds understanding automatically in the background.

3. Problem Statement

<i>"Despite daily consumption of substantial digital learning content, individuals lack an</i>
<i>automated, intelligent system capable of passively capturing that content, extracting</i>
<i>structured knowledge from it, identifying semantic relationships across time, and enabling</i>
<i>conversational retrieval — resulting in chronic knowledge loss, fragmented understanding,</i>
<i>and failure of learning to compound over time."</i>

4. What We Are Building

Passive Second Brain is an AI-powered personal knowledge management system that operates entirely in the background of a user's digital life. The system consists of four tightly integrated layers:

4.1 Layer 1 – Passive Capture Engine

A lightweight Chrome browser extension and optional desktop agent monitors the user's learning activity in real time. It captures:

- ▶ Webpages where reading time exceeds a defined threshold (configurable, default 60 seconds)
- ▶ YouTube video transcripts where watch completion exceeds 50%
- ▶ Locally opened PDFs and documents via the desktop agent
- ▶ Optional voice notes via microphone transcription using Whisper

All captured data remains locally stored on the user's device. No raw content is transmitted to any external server, ensuring complete data privacy.

4.2 Layer 2 – Nightly AI Processing Pipeline

At a user-defined time each night, an automated pipeline processes all content captured during the day. Using large language models via Groq API (Llama 3.3 70B), the pipeline performs the following:

- ▶ Extracts key concepts, entities, and factual claims from raw content
- ▶ Identifies semantic relationships between new and existing knowledge nodes
- ▶ Generates vector embeddings stored in ChromaDB for similarity-based retrieval
- ▶ Produces a structured 'Today's Learning Digest' summarising what was absorbed
- ▶ Recalculates forgetting scores for all existing nodes in the graph

4.3 Layer 3 – Semantic Knowledge Graph

All extracted knowledge is stored as an interconnected graph in Neo4j. Each concept is a node; relationships between concepts are directed edges with semantic labels such as 'is a subset of', 'contradicts', 'is prerequisite for', and 'extends'.

A forgetting score is computed for each node using the SM-2 spaced repetition algorithm – the same algorithm underlying Anki – adapted for graph nodes rather than flashcards. When a node's score crosses a critical threshold, the system proactively surfaces it in the user's dashboard.

4.4 Layer 4 — Conversational Interface & Visual Dashboard

The React-based frontend provides the complete user-facing experience:

- ▶ **Natural Language Chat Interface** — User asks questions in plain English; system answers from their own captured knowledge using RAG — not from the internet.
- ▶ **Interactive Visual Knowledge Graph** — Every concept mapped, colour-coded by domain, with node brightness indicating memory freshness. Built with D3.js.
- ▶ **Daily Digest Panel** — What was learned today, what it connects to in existing knowledge, and what is approaching the forgetting threshold.
- ▶ **Privacy Control Panel** — Granular control over what is tracked, paused, or permanently deleted.

5. Product Requirements Document (PRD)

5.1 Product Vision

To build the world's first fully passive, AI-native personal knowledge graph that requires zero user effort and ensures that everything a person learns becomes permanently accessible, interconnected, and unforgettable.

5.2 Target Users

- Final-year engineering students managing large volumes of study material
- Software developers consuming technical documentation, tutorials, and research papers
- Researchers and academics building knowledge across domains over time
- Self-learners who consume educational content across multiple platforms daily

5.3 Functional Requirements

ID	Requirement	Priority	Module
FR-01	Capture webpage content after 60s reading time via browser extension	Must Have	Capture Engine
FR-02	Extract YouTube video transcripts for videos watched >50%	Must Have	Capture Engine
FR-03	Ingest locally opened PDFs via desktop agent	Must Have	Capture Engine
FR-04	Provide one-click pause/resume for all tracking	Must Have	Privacy
FR-05	Run AI processing pipeline on a nightly schedule	Must Have	Pipeline
FR-06	Extract concepts and relationships using LLM	Must Have	Pipeline
FR-07	Store knowledge as graph nodes and edges in Neo4j	Must Have	Knowledge Graph
FR-08	Compute forgetting score per node using SM-2 algorithm	Must Have	Knowledge Graph
FR-09	Generate vector embeddings in ChromaDB for RAG retrieval	Must Have	Pipeline
FR-10	Provide conversational query interface over personal knowledge	Must Have	Frontend
FR-11	Render interactive visual knowledge graph with D3.js	Must Have	Frontend
FR-12	Generate daily learning digest automatically each night	Should Have	Pipeline
FR-13	Surface proactive alerts for fading knowledge nodes	Should Have	Knowledge Graph
FR-14	Filter content against user-defined learning domains	Should Have	Capture Engine
FR-15	Support voice note transcription via Whisper (local)	Could Have	Capture Engine

5.4 Non-Functional Requirements

Category	Requirement
Privacy	All raw captured content stored locally on user device only — never transmitted externally
Performance	Nightly processing pipeline completes within 5 minutes for up to 50 captured items

Performance	Conversational query response returned within 3 seconds
Reliability	Browser extension must not degrade page load performance by more than 5%
Usability	New user can complete setup and see first graph within 10 minutes
Scalability	Knowledge graph must support up to 10,000 nodes without performance degradation
Security	All API keys stored in local environment variables — never in frontend code

5.5 Tech Stack

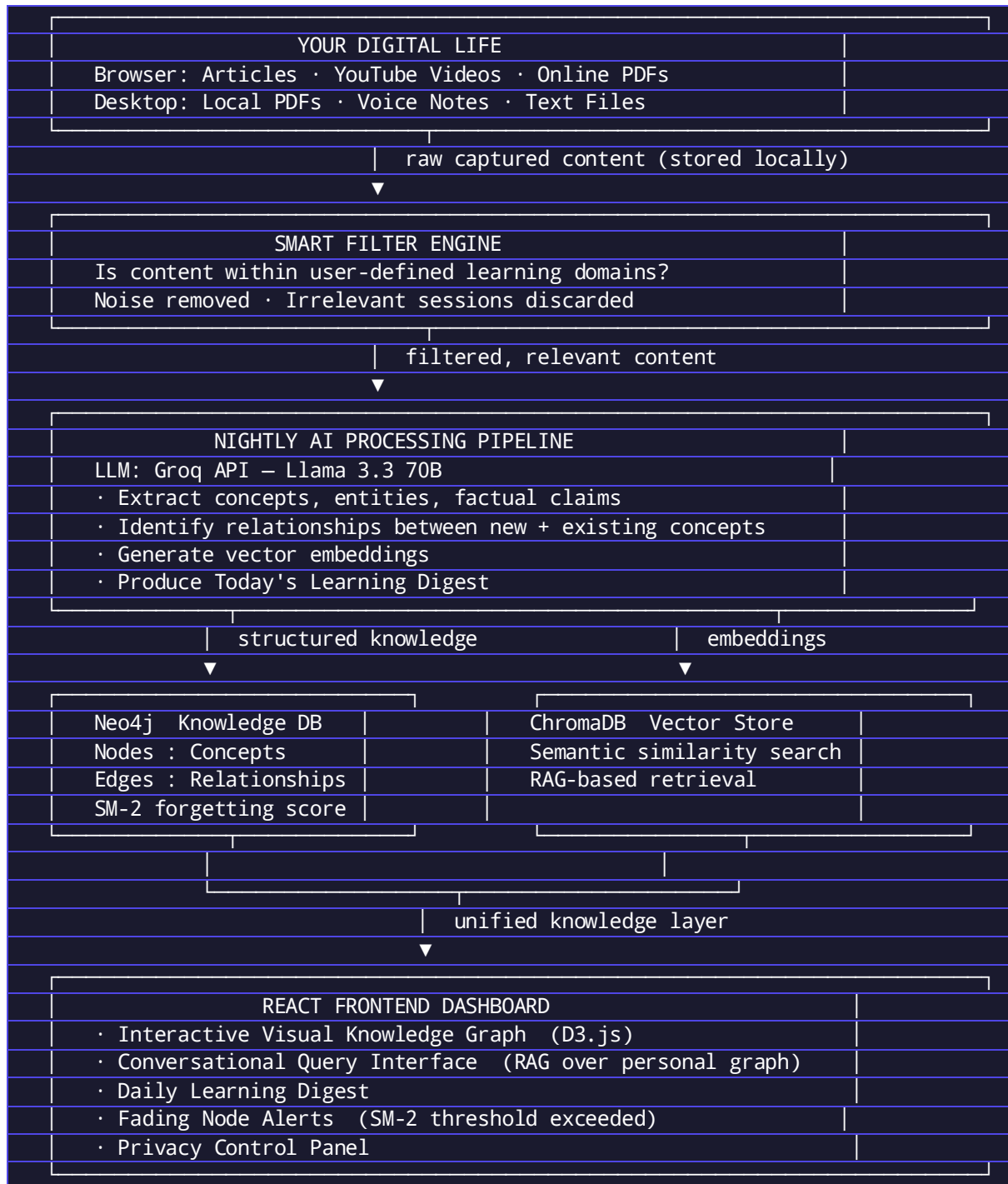
Component	Technology	Justification
Browser Extension	Chrome Extension API (JS)	Universal cross-platform browser integration
LLM Processing	Groq API — Llama 3.3 70B	Fastest free inference; no timeout on large context
Vector Store	ChromaDB	Lightweight, local-first, open-source vector DB
Knowledge Graph	Neo4j	Industry-standard graph DB; rich Cypher querying
Backend / API	FastAPI (Python)	Async support; ideal for pipeline orchestration
Frontend UI	React.js	Component-based; rapid complex UI development
Graph Visualisation	D3.js	Industry standard for force-directed graph rendering
Transcription	OpenAI Whisper (local)	State-of-the-art; runs fully offline for privacy

5.6 Differentiation from Existing Tools

Feature	Notion AI	Obsidian	Anki	Readwise	Passive Second Brain
Zero manual input required	X	X	X	X	✓
Automatic content capture	X	X	X	Partial	✓
Semantic knowledge graph	X	✓	X	X	✓
Forgetting curve tracking	X	X	✓	✓	✓
Conversational recall (RAG)	✓	X	X	X	✓
Visual knowledge map	X	✓	X	X	✓
Privacy-first / local storage	X	✓	✓	X	✓

6. System Architecture & Flow Diagram

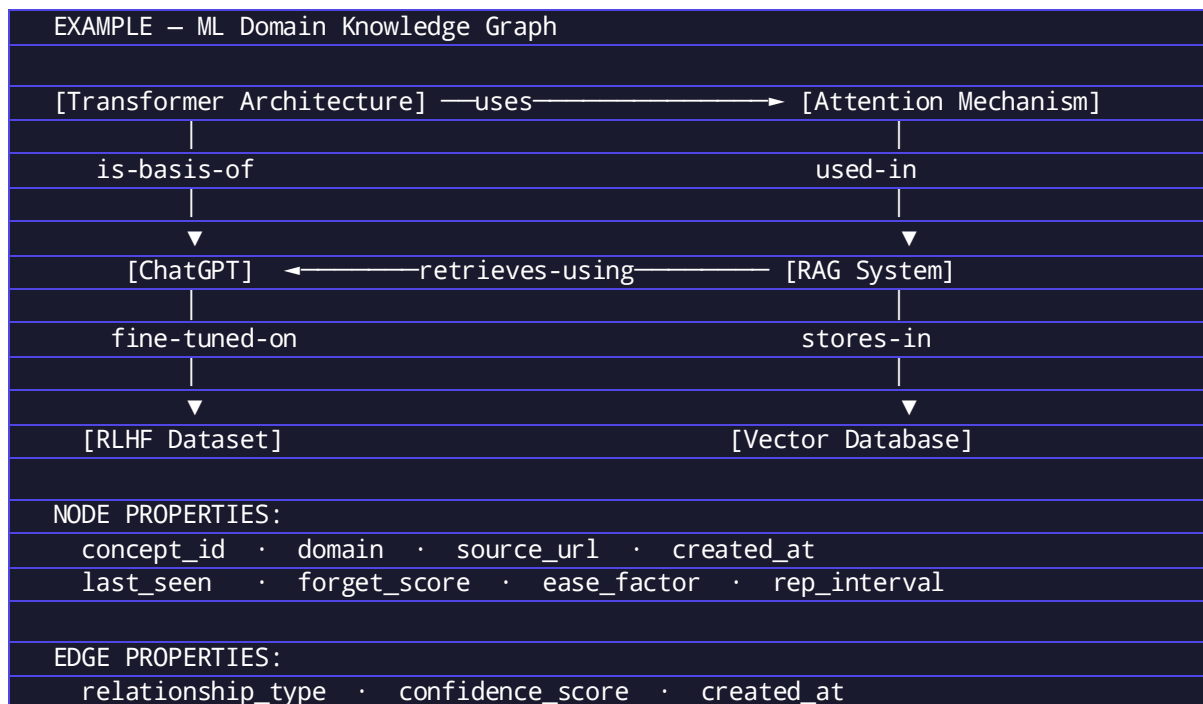
6.1 High-Level Architecture Flow



6.2 Data Flow — Step by Step

Step	Actor	Action	Output
1	Chrome Extension	User browses article for >60s or watches YouTube >50%	Raw HTML / transcript saved locally
2	Desktop Agent	User opens PDF or text file on machine	Raw text extracted and queued
3	Filter Engine	Content checked against user-defined learning domains	Relevant content approved; noise discarded
4	Scheduler	Nightly trigger fires at user-set time (e.g. 11 PM)	Processing pipeline initiated
5	LLM (Groq)	Each content item sent to LLM for concept extraction	Structured JSON of concepts + relationships
6	ChromaDB	Concepts embedded as vectors and stored	Vector index updated
7	Neo4j	New concept nodes and edges written to knowledge graph	Graph updated with new knowledge
8	SM-2 Algorithm	Forgetting scores recalculated for all existing nodes	Nodes marked: fresh / fading / critical
9	LLM (Groq)	Summarises today's additions and cross-connections found	Daily digest ready for user
10	React Dashboard	User opens app, views digest, explores graph, asks questions	Answers retrieved via RAG from personal graph

6.3 Knowledge Graph – Node & Edge Structure



7. Faculty Q&A – Anticipated Questions

The following questions represent typical faculty evaluation criteria for final-year CSE/AI-ML capstone projects:

Q: How is this different from Notion AI or Obsidian?

A: Notion AI and Obsidian require the user to manually save and tag every piece of content. They store knowledge as flat files without semantic understanding of relationships. Passive Second Brain requires zero manual input, automatically extracts structured knowledge, builds a relational graph, and tracks memory decay over time. No existing tool combines all five capabilities in one pipeline.

Q: What is the role of the knowledge graph? Why not just a vector database?

A: A vector database (ChromaDB) enables similarity-based retrieval — finding content semantically close to a query. However, it cannot represent directional relationships between concepts. The knowledge graph (Neo4j) stores this relational structure explicitly — enabling reasoning such as 'Concept A is prerequisite for Concept B'. Both are used together: ChromaDB for retrieval speed, Neo4j for relational reasoning.

Q: How do you handle privacy given that you are monitoring user activity?

A: Privacy is a first-class design constraint. All raw captured data is stored exclusively on the user's local machine. Only processed, abstracted knowledge — concept names and relationships — is handled by the AI pipeline. The user has full visibility into everything captured, can pause tracking instantly, and can delete any content at any time.

Q: What is the technical novelty of this project?

A: The novelty lies in three contributions working as one system: (1) passive, automatic knowledge capture requiring zero user effort, (2) real-time semantic knowledge graph construction from unstructured multi-modal content, and (3) algorithmic implementation of the Ebbinghaus forgetting curve on a personal knowledge graph. Each exists in isolation in prior work — their integration as one cohesive system has not been demonstrated.

Q: What spaced repetition algorithm are you using?

A: A modified SM-2 algorithm — the same algorithm underlying Anki — adapted for graph nodes rather than flashcards. Each node has an easiness factor, repetition interval, and last-reviewed timestamp. The forgetting score is computed as a function of these three variables. When the score crosses a threshold, the system surfaces that concept proactively in the dashboard.

Q: Why Groq API instead of OpenAI or Gemini?

A: Groq offers the fastest free inference available in 2026 — Llama 3.3 70B at 14,400 requests/day on the free tier. Gemini API produced consistent timeout errors on large context payloads during nightly batch processing. Groq's response latency is under 1 second even for large prompts, making it ideal for processing 30–50 captured items per night reliably.

8. Module-Wise Build Plan & Timeline

Phase	Module	Deliverable	Timeline
1	Project Setup	Repo structure, FastAPI skeleton, Neo4j + ChromaDB local setup, environment config	Week 1
2	Capture Engine	Chrome extension capturing URLs, time-on-page, and YouTube transcripts	Week 2–3
3	Filter Engine	Domain classification; noise vs. relevant content filtering logic	Week 4
4	AI Pipeline	Groq LLM integration; concept + relationship extraction from raw text	Week 5–6
5	Knowledge Graph	Neo4j schema design; node/edge creation; Cypher queries for traversal	Week 7–8
6	SM-2 Algorithm	Forgetting score computation per node; proactive alert trigger logic	Week 9
7	RAG Interface	ChromaDB embeddings; conversational query endpoint; FastAPI RAG pipeline	Week 10–11
8	React Frontend	Dashboard UI; D3.js graph visualisation; chat interface; daily digest panel	Week 12–14
9	Integration & Testing	End-to-end pipeline test; performance optimisation; error handling	Week 15
10	Demo & Documentation	Live demo preparation; full project report; viva Q&A preparation	Week 16



Key Advice: Build Phase 1–4 first and get the pipeline working end-to-end

with dummy data before touching the frontend. A working backend that

extracts one concept from one article is more valuable than a beautiful

UI that does nothing. Demo the pipeline first, polish the UI last.